# A Combined Color and CNN Approach for Indoor Navigation on MAVs

M. Sanz Piña, L. Pedretti, T. Calzolari, D. Townsend, E. Bester, H. Kovács
Delft University of Technology, Mekelweg 5, 2628 CD Delft, Netherlands

ABSTRACT

This work presents the development of a lightweight, vision-based navigation system for a micro aerial vehicle operating in a cluttered indoor environment. Several motion-based, appearance-based, and learning-based perception strategies were rapidly prototyped in Python and OpenCV. Based on their performance and computational cost, an appearance-based obstacle avoidance pipeline was selected for onboard deployment. The final method operates directly on the YUV camera stream and combines color segmentation with simple structural validation to detect orange pillars and green plants in real time at 20 Hz. In parallel, a compact convolutional neural network (CNN) was developed for gate detection, achieving reliable performance in offline tests, though not fully integrated before competition flight. The results demonstrate that classical color-based methods remain effective for real-time obstacle avoidance on resource-constrained MAVs, while lightweight CNNs offer a promising solution for structured target detection such as gates.

## 1 Introduction

Autonomous navigation in indoor environments is challenging for micro aerial vehicles (MAVs), which must interpret their surroundings and make safe decisions under strict computational constraints. Vision is an appealing sensing modality, offering rich information at low payload cost, but it requires methods that remain robust to motion blur, viewpoint changes, and partial occlusions.

The work covers the preliminary exploration of different approaches, the design of the final onboard method, and its evaluation in the target environment.

## 2 Problem Definition

The objective of this project is to enable a Parrot Bebop Drone to navigate autonomously in an indoor environment using only its onboard sensors. The platform provides front- and downward-facing monocular cameras, an IMU, and a stabilization module.

Using these sensors, the MAV must detect obstacles, identify free and occupied regions, and generate safe waypoints in real time. A secondary objective is to detect a gate structure and steer toward it to achieve a successful pass-through.

System performance is evaluated in a competition setting, based on distance traveled, number of gate traversals, and number of collisions.

## 3 Related Work

Several approaches to vision-based navigation have been proposed in the literature, which can be broadly classified into motion-based, appearance-based, and learning-based methods [1].

Motion-based approaches, such as optical flow, are widely used for indoor navigation. However, their performance strongly depends on image quality and can degrade under motion blur. In addition, they often require feature tracking and more complex computations, increasing the computational load [2]. Detecting objects from a moving camera is also particularly challenging because of rapidly changing backgrounds and camera motion, which limit the reliability of motion cues alone [3].

In contrast, appearance-based methods rely on visual features such as color and shape to detect relevant elements in the scene. These methods are generally simpler and better suited for real-time applications on resource-constrained platforms [4].

Learning-based approaches have also been explored for MAV navigation. Although they can improve perception performance, they often require significantly more computation and may be unsuitable for low-powered onboard systems without careful optimization [5].

## 4 Preliminary Tested Methods

Several preliminary approaches for gate and obstacle detection were explored during system development, consistent with Section 3.

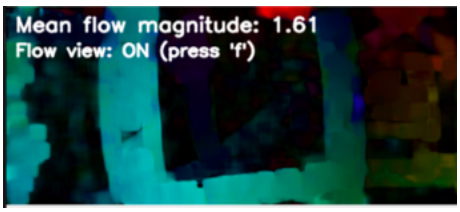
### *4.1 Motion-Based Approach*

Dense optical flow was first explored using Farnebäck's method [6] in Python and OpenCV. Although obstacles produce characteristic divergence patterns, the method required continuous motion and became unreliable under motion blur, making it unsuitable for stable onboard use.

Sparse feature tracking was also tested using FAST [7], GFTT [8], and ORB [9] features with pyramidal Lucas–Kanade tracking [10]. While more stable than dense flow, it suffered from the same fundamental limitation: meaningful

Code available at: https://github.com/tommasocalzolari/paparazzi.git, branch compressed_detector
Pull request: https://github.com/tudelft/paparazzi/pull/119

cues only appeared when the drone moved, and motion blur or low texture caused rapid feature loss. Both approaches were therefore discarded.




(a) Dense optical flow method.


(b) Sparse optical flow method.

Figure 1: Comparison of dense and sparse optical flow for motion-based detection.

### *4.2 Appearance-Based Methods*

Appearance-based methods were explored because the obstacles have distinctive colors, making them suitable for real-time MAV navigation.

#### 4.2.1 Orange detection

The initial experiments for orange obstacle detection were based on HSV color segmentation using OpenCV, enabling rapid prototyping on recorded sequences.

A fixed threshold was applied to isolate orange pixels, followed by morphological filtering (opening and closing) to reduce noise and improve mask consistency. Contours were then extracted, and bounding boxes were drawn around sufficiently large regions. Figure 2 shows an example of the resulting detection using this approach.

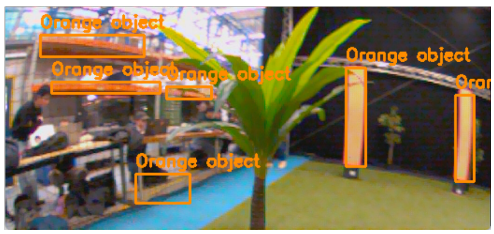



Figure 2: Initial orange detection using HSV thresholding.

While this approach allowed quick validation, it was sensitive to illumination changes and produced false positives in similarly colored regions. Moreover, the reliance on HSV conversion and OpenCV operations made it less suitable for efficient onboard deployment.

These limitations motivated the transition to a YUV-based implementation with additional spatial constraints (Section 5).

#### 4.2.2 Green Detection

The initial plant detection approach relied also on HSV color thresholding using OpenCV, followed by morphological filtering to reduce noise and improve mask consistency.

However, this method proved insufficient in the CyberZoo environment, where the floor exhibits a similar green appearance. As a result, the raw green mask included both plants and large portions of the ground.

To address this, connected-component analysis was applied to the binary mask [11]. Each region was evaluated using simple geometric cues: regions touching the bottom of the image or spanning large horizontal areas were classified as ground, while the remaining ones were considered potential obstacles.


Figure 3: Initial green detection using HSV thresholding and connected components.

As shown in Figure 3, this approach partially reduced confusion between floor and plants, but remained sensitive to scene structure. In particular, interruptions such as gates could fragment the floor into smaller regions, leading to false obstacle detections.

These limitations indicate that color and region-based cues alone are insufficient for robust plant detection, motivating the integration of structural information (e.g., edges) in the final system.

#### 4.2.3 Gate Detection Exploration

Two main visual cues were explored for gate detection: blue color segmentation and the checkered corner pattern. A reliable first step was blue detection in YUV space, shown in Figure 4. Since this blue did not appear elsewhere in the environment, thresholding allowed a relatively consistent extraction of gate regions.

A second approach targeted the checkered pattern on the gate corners by detecting saddle-point-like structures [12] using diagonal difference kernels of different sizes and the Sobel operator. Among the tested options, the 4x4 kernel

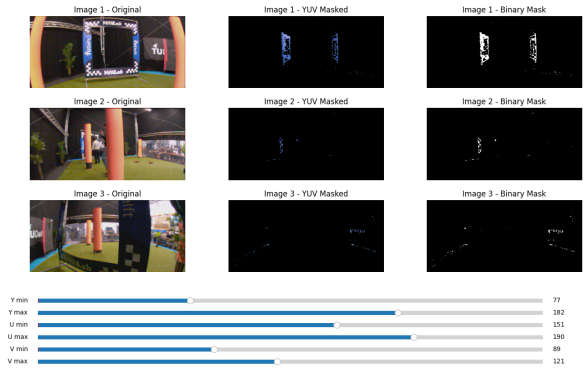
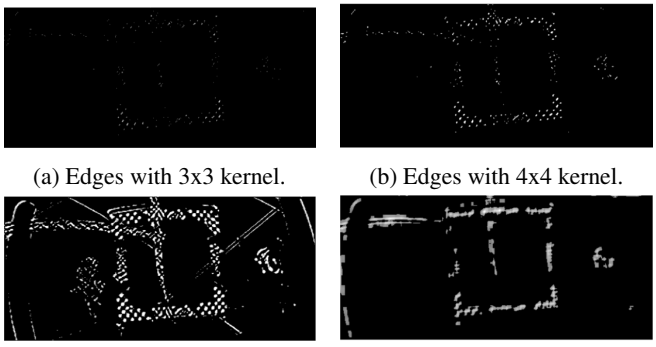



Figure 4: YUV blue threshold tuner.

gave the best trade-off between sensitivity and robustness, as shown in Figure 5. The kernel response was binarized using a threshold proportional to the maximum intensity; a value of 0.9 reduced grid-like background structures while preserving the gate pattern. However, the resulting mask still contained significant noise, so this method was not sufficiently reliable on its own.



(a) Edges with 3x3 kernel.

(b) Edges with 4x4 kernel.

(c) Edges with 6x6 kernel.

(d) Combined Sobel (9x9).

Figure 5: Checkered pattern detector kernels.

Starting from these cues, several gate-targeting strategies were investigated. A histogram-based method was first considered, with the goal of identifying the two vertical gate sides as peaks in the image histogram. In practice, it proved too sensitive to partial occlusions and noise, generating irrelevant peaks (Figure 6). Restricting the checkered response to regions near the blue mask reduced some noise, but at too high a computational cost. A more promising alternative used a downscaled blue mask and estimated the gate direction from the centroid of blue pixels, averaged over several timesteps for robustness to temporary occlusions (Figure 7). Due to time and constraints on integrating into the existing framework, this method was not deployed on the drone for the competition.

For integration with the existing orange and green obstacle detection framework, the final explored solution treated

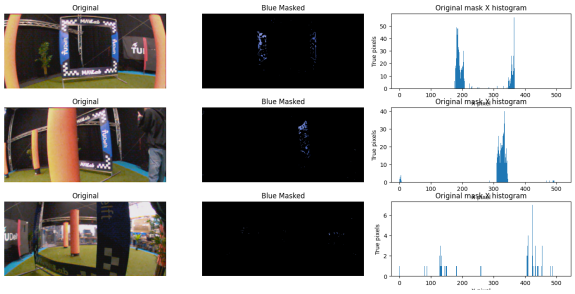
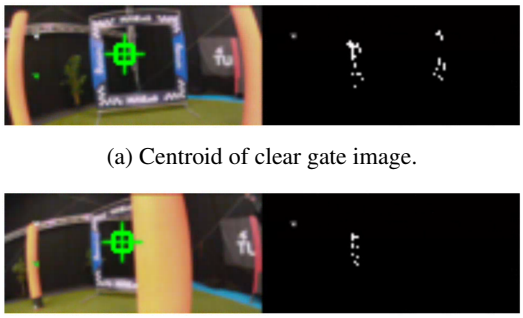



Figure 6: Histogram of blue pixels within image including normal, occluded, and noisy images.

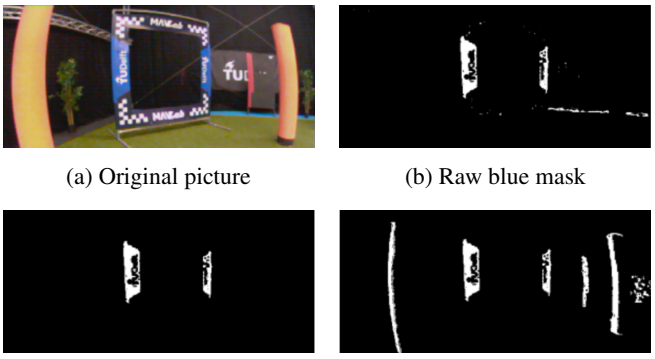


(a) Centroid of clear gate image.

(b) Centroid of gate during temporary occlusion.

Figure 7: Comparison of centroid detection performance.

blue regions as image objects to be validated. Connected blue pixels were grouped into patches, enclosed by bounding boxes, and filtered based on size, aspect ratio, and blue-pixel density. The pixels inside the validated boxes formed the valid blue mask shown in Figure 8, whose blue occupancy in each image subsection could then be passed to the navigation module. Although this approach fit well within the existing framework, it was not implemented for the competition because testing time was insufficient for tuning and validation.



(a) Original picture

(b) Raw blue mask

(c) Validated blue mask

(d) Combined masks

Figure 8: Processing stages of the blue detector.

Code available at: https://github.com/tommasocalzolari/paparazzi.git, branch compressed_detector
Pull request: https://github.com/tudelft/paparazzi/pull/119

### 4.3 *Learning-Based Methods*

Learning-based approaches, such as monocular depth estimation and CNNs were also explored to assess whether lightweight neural networks could provide more robust perception than classical methods.

#### 4.3.1 Monocular Depth Estimation

To address obstacle detection, monocular depth estimation was tested using MiDaS-based models [13] as seen in Figure 9. The small model struggled with overexposed regions particularly the left side of the image, where brightness often washed out object structure. The larger MiDaS v2.1 model (384 resolution) produced smoother depth maps, but failed to detect thin structures such as trees consistently. Both models were also computationally heavy for onboard deployment, making them unsuitable for real-time navigation.

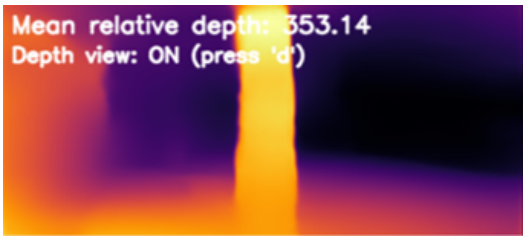



Figure 9: Initial test of MiDas monocular depth estimation.

#### 4.3.2 CNN Approach for Gate Detection

In designing the CNN, the goal was to train a very small and efficient CNN on images containing the gate and use it to estimate its approximate position in pixel coordinates, so that the MAV could be guided toward the gate while still performing obstacle avoidance [14, 15, 16].

The development pipeline followed the standard sequence of dataset acquisition, labeling, model training, and final evaluation. For this task, a large dataset was collected using the MAV's onboard camera, with particular focus on the gate.

Since the model had to run onboard, the network was designed to be extremely small and lightweight. The image was shrunk to a compact input tensor of size $3 \times 12 \times 48$, with channels corresponding directly to the normalized $Y$, $U$, and $V$ components of the incoming UYVY camera stream. The architecture consisted of three $(3 \times 3)$ convolutional layers with same padding and increasing channel depth, $3 \rightarrow 8$, $8 \rightarrow 16$, and $16 \rightarrow 24$, each followed by a ReLU activation and max-pooling layer. Then, a fixed average-pooling operation produced a flattened 144-dimensional feature vector. The prediction head consists of three fully connected layers of dimensions $144 \rightarrow 64$, $64 \rightarrow 32$, and $32 \rightarrow 11$, with ReLU activations after the first two layers.

The initial version of the CNN was designed to predict the positions of the four gate corners, together with the center position and a confidence score. Its output was therefore an 11-dimensional tensor (five sets of $u$ and $v$ pixel coordinates and the confidence score). This first model was trained on a relatively small subset of roughly 1000 manually labeled gate images, annotated by team members using a custom Python-based labeling framework shown in Figure 10.

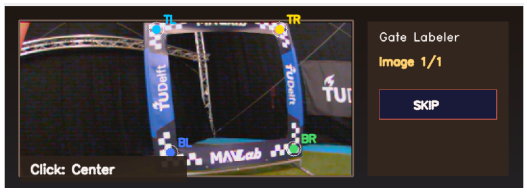



Figure 10: Custom Python-based labeling framework used to annotate gate images for the CNN dataset.

This first solution performed poorly on unseen images, with predicted positions often far from the true gate location. The approach was therefore revised, as described in the next section.

## 5 Final System Implementation

Based on the constraints discussed in the previous section and on the preliminary test results, the final onboard system was designed as a lightweight appearance-based pipeline. It operates directly on the YUV camera stream and combines color segmentation with simple structural validation to detect navigation-relevant obstacles in real time.

A gate detection pipeline was also developed and validated separately through real flight experiments. However, the fully integrated system, combining gate detection and obstacle avoidance, was only tested onboard to a limited extent before the competition, and thus, not enabled during the competition flights.

The final system, shown in Figure 11, consists of a vision module that processes the camera stream and computes a forward risk map, and a navigation module that uses this map to select safe motion directions.

### 5.1 *Vision Module*

The vision module processes images from the monocular camera to generate a risk map, where regions corresponding to obstacles are identified. The module consists of three main processing stages.

#### 5.1.1 Pre-Processing Images

Before detection, each frame is adapted to the reference format assumed by the pipeline. Specifically, the onboard camera provides an image rotated $90^\circ$ clockwise. To account for this, the image is handled in a logical rotated coordinate system. Rather than rotating the full image buffer explicitly,

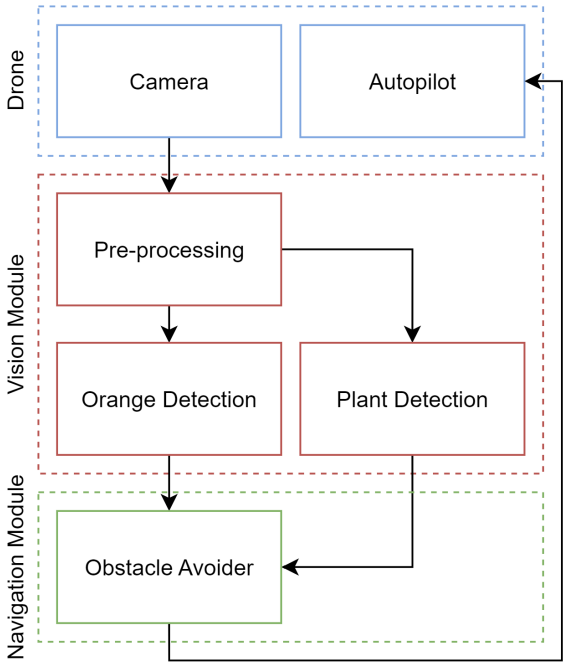



Figure 11: Architecture of implemented obstacle detection and avoidance system.

the code remaps coordinates when pixels are accessed. This choice avoids an additional memory copy and keeps the preprocessing lightweight.

In addition, as the relevant obstacle structure remains visible at lower resolution, the input image was downsampled by two before analysis. This reduces the number of processed pixels per frame without significantly affecting detection quality. As a result, all subsequent stages operate on a smaller image and achieve lower computational cost. This step was also essential for real-time operation, as shown in Section 6.

### 5.1.2 Orange Detection

The orange branch starts by thresholding the image in YUV color space. A pixel is classified as orange if its luminance and chromatic components lie within predefined ranges and if an additional chromatic difference condition is satisfied, producing a binary orange mask.

To improve robustness, the detection is restricted to a region of interest covering the lower part of the image, where nearby obstacles are most relevant for navigation.

This mask is then processed within a region of interest focused on the lower part of the image. The image is scanned through narrow vertical bands, and only bands containing enough orange pixels arranged in sufficiently long vertical runs are retained. Neighboring valid bands are then merged and filtered using geometric and density constraints.

(a) Original image

(b) Raw orange mask

(c) Validated orange mask

Figure 12: Orange detection pipeline. From top to bottom: original image, raw mask after thresholding, and validated mask after spatial filtering.

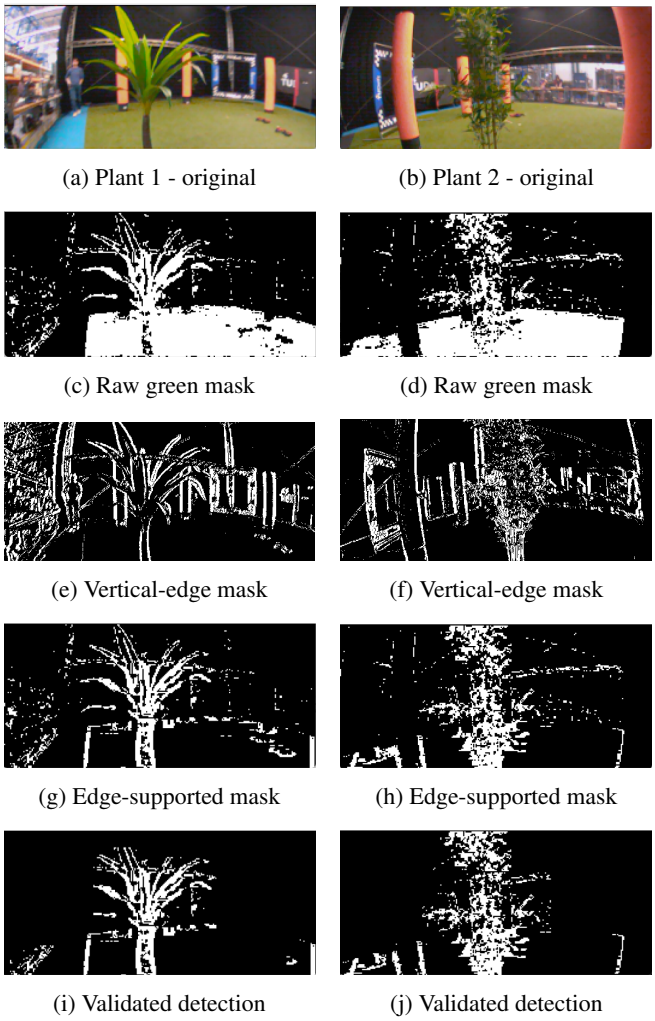
This produces a validated orange mask representing potential cylindrical obstacles.

### 5.1.3 Plant Detection

The green branch also begins with a binary mask obtained by thresholding the image in YUV space.

This raw mask (Figure 13c, 13d) is refined using structural information extracted from the brightness channel Y. A horizontal gradient is used to highlight vertical edges (Figure 13e, 13f), and only green pixels located close to such edges are retained, resulting in a supported green mask (Figure 13g, 13h).

Connected supported regions are used as seeds for candidate obstacles, while very small regions are discarded as noise. Around each seed, the local distribution of raw green pixels is analyzed to estimate the extent of the object. Candidate regions are then validated through constraints on size, vertical extent, aspect ratio, and pixel density, resulting in the validated green mask (Figure 13i, 13j).



(a) Plant 1 - original (b) Plant 2 - original

(c) Raw green mask (d) Raw green mask

(e) Vertical-edge mask (f) Vertical-edge mask

(g) Edge-supported mask (h) Edge-supported mask

(i) Validated detection (j) Validated detection

Figure 13: Plant detection pipeline for two representative plant types.

### 5.2 *Navigation Module*

The navigation module interprets obstacle information from three image sectors (left, middle, right). Each sector maintains an accumulator that increases when obstacles are detected and decays otherwise. A sector is only considered occupied once its accumulator exceeds a threshold, preventing reactions to brief or noisy detections.

Navigation decisions are governed by a finite-state machine with six states shown in Figure 14. In the safe state, the MAV advances along its current heading. Lateral obstacles trigger small heading corrections, while a central obstacle causes the system to rotate until a free direction is found. The out-of-bounds state ensures that generated waypoints remain within the allowed flight area.

Forward motion is modulated by a free-space confidence value that increases when no obstacles are confirmed and decreases otherwise. The step size is scaled by this confidence

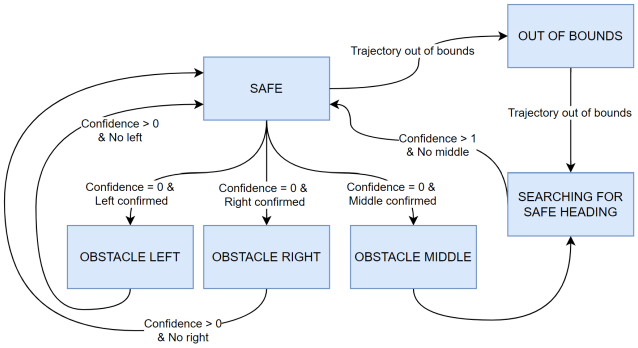



Figure 14: State transition diagram for the navigation module.

up to a fixed maximum, allowing faster progress in clear scenes and more cautious motion in cluttered ones. Overall, the module remains lightweight while providing stable, reactive navigation in real time.

### 5.3 *Gate Detection Module*

The problem of detecting and passing through the gate was addressed separately from obstacle detection. A functional gate detection pipeline was successfully developed and validated as a standalone module.

The final CNN retained the same overall structure as the previous version (see Section 4.3.2), with three convolutional layers and three linear layers, but the output representation was changed. Predicting the four gate corners proved too difficult, mainly because all four corners were often not visible at the same time during labeling. For this reason, the corner-based output was replaced by a 2D bounding-box representation. The final model predicts five values: a gate presence score and the normalized bounding-box parameters $(c_x, c_y, w, h)$. In this way, the network behaves as a single-object detector that estimates whether a gate is present and, if so, its approximate location and size in the image.

The labeling and training pipeline was also revised. Since the initial set of roughly $1000$ manually labeled images was too small, but manually labeling the full dataset would have taken too much time, Ultralytics YOLO11 Nano was used for automatic labeling. The manually labeled images were used to train YOLO, which was then applied to label roughly $15000$ images. Cross-checks showed that the predicted bounding boxes and centers were sufficiently accurate as can be seen Figure 15.

This resulted in a much larger and more representative dataset. Training, validation, and test splits were then created while keeping a similar proportion of gate images in each split. The final model was trained directly on the YUV422 images, using a batch size of $64$, learning rate $10^{-3}$, weight decay $10^{-4}$, AdamW optimizer, and seed $42$. After $60$ epochs, the model from epoch $56$, which achieved the lowest validation loss of $0.0909$, was selected. The total loss was defined as binary cross-entropy with logits for gate presence

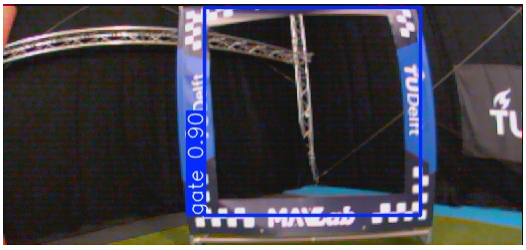



Figure 15: YOLO generated bounding box label.

plus a smooth $L_1$ regression loss for geometry on positive samples:

$$\mathcal{L}_{total} = \mathcal{L}_{BCE} + 5.0\,\mathcal{L}_{reg}.$$

A lightweight prediction module was then implemented to process the incoming image and send the predictions to a dedicated gate navigation module. For standalone testing, this navigation module was designed as a simple state machine with four phases: searching for the gate by rotating, aligning the drone to center the gate in the image, flying toward it, and finally performing a short blind pass-through once the gate was close enough. The logic was based on the detected horizontal gate position, bounding-box size, and detection quality.

The final network was in fact very small, with a maximum channel width of 24, approximately $1.6 \times 10^4$ trainable parameters, and an inference cost on the order of $10^5$ to $10^6$ operations per frame. To support fast computation, the convolutional and linear operations were implemented manually in C rather than through an external inference framework such as PyTorch.

## 6 Results

This section presents the main results of the proposed system, including computational performance, obstacle detection refinement through the color-edge masks, and gate detection experiments, including the CNN-based approach.

### *6.1 Computational Performance*

Real-time onboard operation was a key design requirement. Different implementations of the same vision pipeline were therefore compared in terms of computational efficiency. Since the detection logic and color masks were unchanged, the visual output remained the same, and only the execution time varied. Table 1 summarizes the resulting processing frequencies.

Restricting the analysis to a region of interest already improved performance over the full-resolution version, but the largest gain was achieved with image downsampling. The final implementation reached about 20 Hz, compared with about 1.5 Hz at full resolution and about 3 Hz with ROI-only processing. This improvement was essential for real-time onboard operation.

Table 1: Processing frequency of the vision pipeline for different implementations.

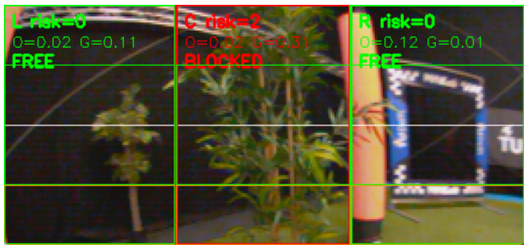
| Configuration | Frequency [Hz] |
|---|---|
| Full resolution | 1.5 |
| ROI only | 3 |
| Downsampled image | 20 |

### *6.2 Obstacle Avoidance Results*

The validated orange and green masks were combined into a single representation, which was then used to generate a risk mask for the navigation module. This risk mask encodes the spatial distribution of obstacles and allows the drone to adjust its trajectory accordingly. Figure 16 illustrates the final processing pipeline. Starting from the original image, the combined mask highlights the detected obstacle regions, and the risk mask providing a more structured representation used for avoidance decisions.

(a) Original image

(b) Combined mask




(c) Risk map

Figure 16: Final pipeline.

Experimental tests showed that the system was able to successfully avoid both orange and green obstacles in most scenarios. However, the performance was sensitive to parameter tuning. In particular, adjusting the thresholds on-site sig-

Code available at: https://github.com/tommasocalzolari/paparazzi.git, branch compressed_detector
Pull request: https://github.com/tudelft/paparazzi/pull/119

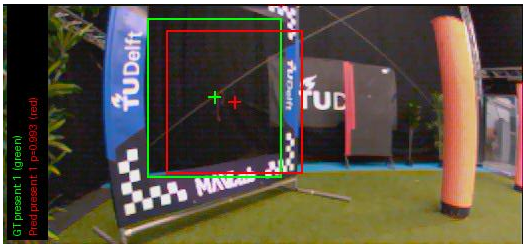
nificantly improved detection reliability under varying lighting conditions. If the Y channel was not properly tuned, the drone exhibited overly cautious behavior, as obstacles were detected as too close.

Additionally, since the blue gate detector was not included in the final implementation, collisions with gates could still occur. This highlights a limitation of the current system, which relies on partial color-based detection and does not yet cover all relevant obstacle types.

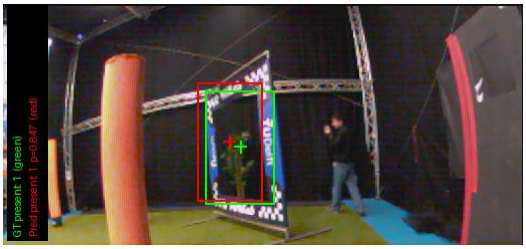
These observations are reflected in the system's performance during the final competition. Overall, the results demonstrate that the proposed lightweight vision pipeline is effective for real-time obstacle avoidance, while remaining computationally efficient. In the final competition, the system achieved a distance of 69 m, ranked 4th out of 14 teams, successfully passed through the gate four times but also experienced two crashes with the gate. However, these results confirm the practical viability of the system in a competitive real-world setting.

### *6.3 CNN Results*

The final CNN achieved the following results on the unseen test set: a total test loss of $0.1121$, composed of a binary cross-entropy loss of $0.0996$ and a regression loss of $0.0025$. In addition, the final test presence accuracy reached $96.09\%$, considering a margin of 20 pixels, as shown in Figure 17.




(a) Prediction with $0.99$ confidence




(b) Prediction with $0.84$ confidence from a skewed angle

Figure 17: CNN detections on the test set (unseen images). Ground-truth gate centers and bounding boxes are shown in green, while the predicted ones are shown in red.

When deployed on the actual drone, the model achieved a processing time of approximately $5$ ms per image, corresponding to a throughput of roughly $200$ Hz. During real-world testing, the drone was able to detect the gate, fly toward it, and perform the blind pass-through maneuver successfully.

## 7 Conclusion and Future Work

This work presented the development of lightweight vision-based navigation solutions for a micro aerial vehicle operating in a cluttered indoor environment. For obstacle avoidance, a classical computer vision pipeline based on YUV color filtering and structural validation was implemented and successfully deployed onboard, achieving real-time performance at about 20 Hz. In parallel, a lightweight CNN-based gate detector was developed and validated as a standalone module, reaching about 5 ms inference time per image and enabling reliable gate-directed navigation. Although both modules showed promising results independently, limited testing time prevented their complete integration into a single unified system before the competition. This remained the main limitation of the final project.

Overall, the results show that both classical vision methods and compact learning-based models can be effective on resource-constrained MAV platforms when carefully tailored to the task. In particular, the project suggests that a tighter integration of the CNN-based gate detector with the obstacle avoidance pipeline could provide a more capable and versatile navigation system, combining safe reactive flight with reliable gate targeting. Future work should therefore focus first on the full onboard integration of the two modules and on more extensive joint flight testing. A further promising direction is the introduction of adaptive color-threshold tuning to compensate for changes in illumination. Similar ideas have been explored in robotics through online color map learning and auto-adjusting vision systems under changing lighting conditions [17].

## References

[1] Sankarasrinivasan S and Balasubramanian Esakki. Vision based algorithms for mav navigation. In *2015 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, pages 1–4, 2015.

[2] Masoud Mohtadifar and Hadi Seyedarabi. Autonomous, bio-inspired vision-based navigation system for indoor flying using hybrid optical flow and stereopsis methods. In *2022 30th International Conference on Electrical Engineering (ICEE)*, pages 191–197, 2022.

[3] Artem Rozantsev, Vincent Lepetit, and Pascal Fua. Detecting flying objects using a single moving camera. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(5):879–892, 2017.

[4] Lim-Kwan Kong, Jie Sheng, and Ankur Teredesai. Basic micro-aerial vehicles (mavs) obstacles avoidance using monocular computer vision. In *2014 13th Interna-*

*tional Conference on Control Automation Robotics Vision (ICARCV)*, pages 1051–1056, 2014.

[5] Khushal Brahmbhatt, Akshatha Rakesh Pai, and Sanjay Singh. Neural network approach for vision-based track navigation using low-powered computers on mavs. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 578–583, 2017.

[6] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the 13th Scandinavian Conference on Image Analysis*, pages 363–370. Springer, 2003.

[7] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. In *European Conference on Computer Vision (ECCV)*, volume 1, pages 430–443. Springer, 2006.

[8] Jianbo Shi and Carlo Tomasi. Good features to track. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 593–600. IEEE, 1994.

[9] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary R. Bradski. Orb: An efficient alternative to sift or surf. In *International Conference on Computer Vision (ICCV)*, pages 2564–2571. IEEE, 2011.

[10] Jean-Yves Bouguet. Pyramidal implementation of the lucas-kanade feature tracker: Description of the algorithm. Technical Report 5, Intel Corporation, 2001.

[11] Lifeng He, Xiwei Ren, Qihang Gao, Xiao Zhao, Bin Yao, and Yuyan Chao. The connected-component labeling problem: A review of state-of-the-art algorithms. *Pattern Recognition*, 70:25–43, 2017.

[12] Luca Lucchese and Sanjit K. Mitra. Using saddle points for subpixel feature detection in camera calibration targets. *Asia-Pacific Conference on Circuits and Systems*, 2:191–195 vol.2, 2002.

[13] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17582–17592, 2020.

[14] Antonio Loquercio, Ana I. Maqueda, Carlos R. del Blanco, and Davide Scaramuzza. Dronet: Learning to fly by driving. *IEEE Robotics and Automation Letters*, 3(2):1088–1095, 2018.

[15] Alexandros Kouris and Christos-Savvas Bouganis. Learning to fly by myself: A self-supervised cnn-based approach for autonomous navigation. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, page 1–9. IEEE Press, 2018.

[16] Aldrich A. Cabrera-Ponce, Leticia Oyuki Rojas-Perez, Jesus Ariel Carrasco-Ochoa, Jose Francisco Martinez-Trinidad, and Jose Martinez-Carranza. Gate detection for micro aerial vehicles using a single shot detector. *IEEE Latin America Transactions*, 17(12):2045–2052, 2019.

[17] Mohan Sridharan and Peter Stone. Color learning on a mobile robot: Towards full autonomy under changing illumination. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.